

Pre-Impact Fall Detection on a Low-Cost Microcontroller

A cross-dataset generalisation and TinyML deployment study

Full experimental plan, revision 2 — undergraduate conference paper track · 22 August 2026 · Hardware budget: BDT 3,000

Changes from revision 1: SisFall Enhanced replaces the original SisFall as the primary corpus · UP-Fall removed entirely · blanket 38-fold LOSO replaced with a tiered cross-validation strategy

1. Project summary

We train a compact deep learning model to detect a fall from a single waist-worn inertial sensor *before body-ground impact*, evaluate it under a leave-one-dataset-out protocol that most published work avoids, and deploy the quantised model on an ESP32 microcontroller costing under BDT 500, reporting measured on-device latency, memory and battery life.

The constraint driving every choice below is that a BDT 3,000 budget cannot fund research-grade data collection. All training and evaluation data comes from established public benchmarks; the hardware exists to prove real-time feasibility, not to generate the dataset. State this explicitly in the paper — it is a deliberate methodology, not a limitation to apologise for.

1.1 The three contributions

#	Contribution	Claim	Why it is not already done
C1	Leave-one-dataset-out (LODO) generalisation	Report the honest performance collapse when a fall detector meets an unseen dataset, then close the gap with a lightweight domain-adaptation step.	Within-dataset accuracy is saturated near 99%, but published benchmarking shows performance degrades considerably once evaluation extends to other datasets. Most papers never leave their training dataset.
C2	Pre-impact detection	Detect the fall during descent and report <i>lead time</i> before impact, not just post-hoc accuracy.	Requires frame-level temporal labels, available in only two sources (SisFall Enhanced and KFall). Most public datasets support post-fall detection only, so the sub-literature is far smaller.
C3	Verified TinyML deployment	INT8 model running on ESP32 with measured latency, peak RAM, model size and battery life — not laptop-simulated numbers.	Nearly all fall-detection papers report inference on desktop GPUs. Papers combining LODO evaluation and real MCU measurements are rare.

One-sentence pitch: "We show that a 25k-parameter separable CNN, quantised to INT8 and running in under 50 ms on a BDT 450 microcontroller, retains usable pre-impact fall detection accuracy on datasets it was never trained on."

2. What the data physically is

Every dataset here is **plain text files containing numbers**. There is no XML anywhere. Some contain video, which we ignore entirely. What the model consumes is a two-dimensional array: rows are time samples, columns are the six IMU channels.

Dataset	On-disk format	Approx. download	What we extract
SisFall Enhanced	Per-trial text/CSV files. Sensor columns plus a per-sample class column added by the annotators.	~1 GB	6 channels (one accelerometer + gyroscope) and the three-class per-sample label.
KFall	One CSV per trial, named SA06T01R01.csv (subject / task / repetition), plus a separate label file giving fall-onset and impact frame numbers.	a few GB	6 channels from the low-back IMU, plus onset and impact frame indices.
FallAIID	26,420 CSV files across waist, wrist and neck sensors. Accelerometer, gyroscope, magnetometer and barometer columns.	several GB	Waist sensor only, accelerometer + gyroscope. Magnetometer and barometer discarded — our hardware has neither.
UMAFall	CSV traces plus optional video clips. Five body positions.	under 1 GB	Waist channel from the smartphone stream (200 Hz), not the 20 Hz dedicated IMU stream. Videos not downloaded.

2.1 Anatomy of a raw SisFall file

The original files carry no header and no label column — nine comma-separated integers per line, 200 lines per second, representing two accelerometers and one gyroscope in raw ADC counts:

```
-47, -117, -25, -13, -5, 12, -190, -466, -98;  
-49, -115, -22, -11, -3, 14, -196, -458, -102;
```

In the original release the **label lives in the filename**: F01_SA06_R02.txt is fall type 01, young-adult subject 06, repetition 02; D05_SE03_R01.txt is ADL type 05, elderly subject 03. Subject prefixes are SA for the 23 young adults and SE for the 15 older participants.

SisFall Enhanced adds what the original lacks: a per-sample class label, which is exactly what pre-impact work requires.

3. Dataset inventory

Four datasets. All verified publicly accessible in August 2026.

Dataset	Subjects	Position	Native rate	Temporal labels	Access
SisFall Enhanced	38 23 young, 15 older	Waist	200 Hz	Yes — three classes per sample	Kaggle: nvnikhil0001/sisfall-enhanced Original signals: SISTEMIC , sistemic.udea.edu.co/en/research/projects/english-falls/
KFall	32	Low back	~100 Hz	Yes — frame- level onset and impact	sites.google.com/view/kfalldataset Mirror: Kaggle usmanabbasi2002/kfall-dataset
FallAIID	15	Neck, wrist, waist	Config- dependent	No	IEEE DataPort, open access ieee-dataport.org/open-access/fallalld-comprehensive-dataset-human-falls-and-activities-daily-living
UMAFall	17–19	Chest, waist, wrist, ankle, pocket	20 Hz IMU 200 Hz phone	No	Figshare, DOI 10.6084/M9.FIGSHARE.4214283

3.1 Why SisFall Enhanced rather than the original

SisFall Enhanced is the original SisFall recordings re-annotated by Musci et al. (2018) with three per-sample classes: **non-fall** (performing ADLs), **pre-impact fall / alert** (transitioning from a controlled state toward a dangerous one), and **fall** (the state transition leading to impact).

This changes the project materially. Contribution C2 previously rested on KFall alone. With Enhanced, you have **two independent sources of pre-impact labels**, which means you can validate lead time across datasets rather than within one — a much stronger claim — and the contribution survives if KFall's label format proves awkward.

Label quality caveat you must address in the paper. The Musci annotation was produced by defining criteria with domain experts and labelling iteratively, but subsequent authors have criticised the process as necessarily subjective because the labelling was not performed against video references. KFall's temporal labels were designed for this purpose from the outset and should be treated as the more authoritative source.

Practical consequence: report your headline lead-time numbers on *KFall*, and use SisFall Enhanced as the corroborating second dataset. Presenting it the other way round invites a reviewer to attack your primary result on labelling grounds. Acknowledging the asymmetry yourself is far stronger than being caught by it.

3.2 Verification checklist before committing to Enhanced

It is a community-hosted derivative, so confirm the following in week 1 before building anything on top of it:

- All 38 subjects present, with the *SA* / *SE* split intact. If older subjects are missing, you lose the one property that makes SisFall uniquely valuable.
- The sensor values match the original release — still 200 Hz, still raw ADC counts, not pre-filtered or resampled. Diff a handful of files against the SISTEMIC originals.
- The three-class label column is populated on *fall* trials and consistently zero on ADL trials.
- Class boundaries look physically sensible when plotted — the alert interval should precede, not straddle, the acceleration spike.

If any check fails, fall back to the original SisFall for training and rely on KFall alone for C2. Download the original regardless, as your reference copy.

Cite both papers. Using Enhanced obliges you to cite the original dataset (Sucerquia et al., 2017) *and* the annotation work (Musci et al., 2018). Citing only one is the kind of omission that makes reviewers doubt everything else.

3.3 UP-Fall: excluded, and why

Removed from the plan entirely. Two independent reasons, both worth one sentence in the paper:

1. **Sampling rate.** Its accelerometers run at approximately 18.4 Hz, which cannot be raised to our 50 Hz working rate without fabricating signal content. Including it would inflate the measured domain gap with what is really a resampling artifact.
2. **Size.** The full release is on the order of 850 GB, overwhelmingly camera imagery we would never open. Downloading and storing it to extract a stream we then exclude is pure waste.

Explicit exclusion criteria are a small contribution in themselves. Reviewers respect a stated methodological reason far more than silent omission.

3.4 Role assignment

Dataset	Role	Justification
SisFall Enhanced	Primary training corpus + secondary pre-impact source	Largest subject pool, only dataset including genuinely older participants, waist placement matches our device, per-sample temporal labels.

KFall	Primary pre-impact source (C2) + LODO fold	Purpose-built temporal labels of higher confidence than the Enhanced annotations. Low-back placement is close enough to waist to be compatible.
FallAIID	LODO fold + unseen-domain test	Independent hardware, independent lab, waist channel available. Genuinely out of distribution.
UMAFall	Final held-out stress test	Never used in training in any fold. Reserved for a single clean "never seen, never tuned" row in the results table.

4. Preprocessing and harmonisation protocol

This is the highest-risk part of the project. If preprocessing during training differs in any way from preprocessing on the ESP32, the model will score 99% in Python and behave randomly on the device. Freeze this protocol in writing before training anything, and implement it exactly once in a shared module used by both the training scripts and the firmware code generator.

4.1 The seven steps

1. **Channel selection.** Waist / low-back sensor only. Six channels: `ax`, `ay`, `az`, `gx`, `gy`, `gz`. Discard magnetometer, barometer, and SisFall's second accelerometer.
2. **Unit conversion.** SisFall files store raw ADC counts; apply the conversion formulas in its `Readme.txt`. Convert every dataset to acceleration in *g* and angular velocity in *degrees per second*. Verify by plotting a resting trial — the acceleration vector magnitude must sit at 1.0 g.
3. **Anti-alias filtering, then decimation to 50 Hz.** Low-pass at 20 Hz *before* downsampling. Decimating without filtering aliases impact transients into the passband and quietly corrupts your fall class. Use `scipy.signal.decimate`, which filters internally, not array slicing.
4. **Windowing.** 2.0 s windows = 100 samples × 6 channels, stride 0.5 s (50 samples), 75% overlap. Same stride at inference.
5. **Labelling.** Post-fall task: window is positive if it contains the impact. Pre-impact task: window is positive if its right edge lies inside the alert interval — after fall onset, before impact. Both source datasets now supply this directly.
6. **Normalisation.** Compute channel-wise mean and standard deviation on *training subjects only*. Save the twelve numbers to JSON. Apply unchanged to every validation set, test set, and the firmware. Never recompute on test data.
7. **Caching.** Write final windowed arrays to compressed `.npz` and upload as a private Kaggle Dataset. Preprocess once; reuse everywhere.

4.2 Sanity checks before training anything

- Plot one fall and one walking trial from each dataset at 50 Hz on shared axes. They should look qualitatively comparable. An order-of-magnitude amplitude difference means your unit conversion is wrong.
- Confirm resting acceleration magnitude is 1.0 g in every dataset.
- Standardise axis conventions so gravity sits on the same axis with the same sign everywhere. Document the rotation applied to each dataset.
- Count windows per class per dataset and record the imbalance ratios. You need these numbers for the paper.
- Overlay the SisFall Enhanced alert-interval labels on the acceleration trace for ten fall trials and inspect visually. This is your only defence against inheriting an annotation error.

Axis convention is the classic cross-dataset trap. Two labs mounting the same sensor rotated 180° produce data that cannot transfer, and the failure looks exactly like a genuine domain shift. Resolve orientation before concluding anything about generalisation, or you will spend weeks writing up an artifact.

5. Experimental design

5.1 Cross-validation strategy (revised)

Full 38-fold leave-one-subject-out is the gold standard but is wasteful applied to everything. Use a tiered approach: the guarantee that matters is that no subject appears in both train and test, and 5-fold subject-grouped CV provides that at roughly one-eighth the cost.

Where	Protocol	Reason
Baselines (Table I), all ablations (E5)	<code>GroupKFold(n_splits=5)</code> , groups = subject ID	Same no-leakage guarantee, ~8× cheaper. Sufficient for comparing models against one another.
Final proposed model only	Full 38-fold LOSO	Your headline number. Reviewers expect LOSO for the result you are actually claiming.
Cross-dataset (E2)	Leave-one-dataset-out	Dataset is the grouping unit; subject folds do not apply.

E1 — Within-dataset baseline

Protocol: 5-fold subject-grouped CV on SisFall Enhanced for all comparators; full 38-fold LOSO for the proposed model.

Purpose: Prove competitiveness before claiming anything about generalisation. Weak LOSO numbers make weak LODO numbers unpublishable.

Comparators: SMV threshold, SVM on hand-crafted features, Random Forest, 1D-CNN, CNN-LSTM, and the proposed compact model.

Report: sensitivity, specificity, F1, AUC, parameter count — mean and standard deviation across folds.

E2 — Leave-one-dataset-out (Contribution C1)

Protocol: Train on two datasets, test on a third. No fine-tuning, no target labels at any point.

Fold	Train	Test (unseen)

A	SisFall Enhanced + KFall	FallAIID
B	SisFall Enhanced + FallAIID	KFall
C	KFall + FallAIID	SisFall Enhanced
D	All three	UMAFall (held out throughout, reported once)

The headline result is the gap between E1 and E2. Expect a large drop. Report it honestly — the drop *is* the finding, and concealing it is exactly what the existing literature does.

Then close it, in ascending order of cost:

1. **Per-window instance normalisation.** Normalise each window by its own statistics rather than global training statistics. Near-zero MCU cost, often recovers most of the loss. Start here.
2. **CORAL feature alignment.** Aligns second-order feature statistics. No target labels required.
3. **Adversarial domain adaptation (DANN).** Highest cost; attempt only if the first two are insufficient and time permits.

E3 — Pre-impact detection (Contribution C2)

Protocol: KFall as primary (LOSO across its 32 subjects), SisFall Enhanced as the corroborating second dataset.

Primary metric: *lead time* — milliseconds between the model firing and the labelled impact. Report the distribution, not only the mean.

Secondary metrics: sensitivity at a fixed false-alarm budget, and false alarms per hour of ADL data. The second determines whether a device is wearable at all, and most papers omit it.

Operating point: Sweep the decision threshold and present a lead-time-versus-false-alarm curve. A device firing 200 ms early but alarming forty times a day is useless; demonstrating you understand this trade-off distinguishes the paper.

Cross-source agreement: Report lead time on both label sources separately. If they disagree substantially, that discrepancy is itself an interesting finding about annotation methodology — discuss it rather than averaging it away.

E4 — On-device measurement (Contribution C3)

Measurement	Method	Target
Model size (flash)	Size of the <code>.tflite</code> file	< 60 KB
Peak RAM (tensor arena)	TFLite Micro arena high-water mark	< 120 KB
Inference latency	<code>esp_timer_get_time()</code> around invoke, averaged over 1,000 runs	< 50 ms
End-to-end latency	Sensor read to buzzer, via logic analyser or GPIO toggle	< 100 ms
Battery life	Run to shutdown on a charged 1000 mAh cell, continuous inference	Report as measured
Accuracy delta FP32 → INT8	Both evaluated on the same held-out set	< 2 percentage points

E5 — Ablations

- **Window length:** 1.0 / 1.5 / 2.0 s. Shorter means lower latency and less RAM.
- **Sampling rate:** 25 / 50 / 100 Hz. Published work has found lower rates can retain performance while cutting compute.
- **Accelerometer only vs accelerometer + gyroscope.** Dropping the gyroscope saves meaningful power. If accuracy holds, that is a genuine practical result.
- **Sensor placement:** waist vs wrist, using multi-position channels in FallAIID and UMAFall. This is why the BOM includes a second IMU.

6. Model architecture

The MCU memory budget is the design driver, not an afterthought. Target roughly 25,000 parameters, quantising to about 25 KB.

```
Input: 100 x 6 (2.0 s at 50 Hz, 6 IMU channels)
|
+-- Instance normalisation (per-window, per-channel)
|
+-- Conv1D(24, kernel=7, stride=2) + BatchNorm + ReLU    -> 50 x 24
|
+-- SeparableConv1D(48, kernel=5) + BatchNorm + ReLU    -> 50 x 48
+-- MaxPool1D(2)                                         -> 25 x 48
|
+-- SeparableConv1D(64, kernel=3) + BatchNorm + ReLU    -> 25 x 64
|
+-- GlobalAveragePooling1D                               -> 64
|
+-- Dense(32) + ReLU + Dropout(0.3)
+-- Dense(N) + Softmax      N = 2 (post-fall) or 3 (pre-impact)
```

Separable convolutions cut parameter count roughly by an order of magnitude versus standard convolutions at similar accuracy. **Global average pooling** instead of flatten prevents the dense layer from dominating the parameter budget.

With SisFall Enhanced the natural output is **three classes** (non-fall / alert / fall), matching the annotation directly. Train the three-class head as primary and derive binary metrics from it by merging alert and fall — this gives you both result sets from one model rather than training two.

Training configuration: Adam, initial learning rate 1e-3 with cosine decay, batch size 128, up to 100 epochs, early stopping on validation macro-F1 with patience 15. Handle class imbalance with class weighting, not oversampling — oversampling with 75% window overlap leaks near-duplicate windows across the split.

Split by subject, never by window. With overlapping windows, a random split places near-identical windows on both sides and produces accuracy above 99% that means nothing. Every split in this plan is subject-wise or dataset-wise for exactly this reason.

7. Compute environment and budget

Machine	Used for	Notes
MacBook Air M4	Download and preprocessing, exploratory analysis, plotting, quantisation, firmware development and flashing, paper writing, rapid model iteration	With tensorflow-metal it trains these small models at respectable speed. Use it for all debugging and iteration; send only batch runs to Kaggle.
Kaggle	Grouped-CV grids, the single full LOSO run, LODO folds, ablation sweeps	Roughly 30 GPU-hours per week, 12-hour session cap, working directory wiped between sessions.

7.1 Estimated GPU budget

Roughly 200,000 windows from SisFall Enhanced at 50 Hz; about 250 MB in float16, so the whole set fits in RAM. At ~25k parameters the GPU waits on the data pipeline, not on matrix multiplication — expect around 10 s per epoch and roughly 6 minutes per fold with early stopping.

Experiment	Runs	Estimated GPU hours
Table I baselines, 5-fold grouped	6 models × 5	~3
Final proposed model, 38-fold LOSO	38	~4
E2 LODO × 3 adaptation methods	12	~1.5
E3 pre-impact, KFall LOSO + Enhanced	~40	~3
E5 ablations, 5-fold grouped	~50	~5
Total across the whole project		~17 hours

That is roughly half of one week's quota spread across ten weeks. **Compute is not your constraint.** Preprocessing time and debugging time are.

7.2 Workflow rules that protect the budget

- Preprocess locally on the M4. Upload cached .npz windows as a private Kaggle Dataset. Never re-parse raw files inside a GPU session — that is paying GPU rates for CPU work.
- Write results to /kaggle/working/results.csv after every fold, not at the end. A session dying at fold 34 of 38 should not cost you the previous 33.
- Accept a --start-fold argument so a timed-out run resumes rather than restarts.
- Save checkpoints to /kaggle/working/ and download before the session closes.
- Keep a running experiment log — date, config hash, git commit, result — in one file. You will need it when a reviewer asks how a number was produced.

8. Hardware bill of materials

Dhaka market rates, August 2026. Suppliers: TechShopBD, RoboticsBD (Uttara), ElectronicsBD, BDTronics, Daraz.

Component	Qty	Unit (BDT)	Total	Purpose and notes
ESP32 DevKit V1 (30 or 38 pin)	1	410–500	450	Main MCU. 240 MHz dual core, 320 KB usable RAM, 4 MB flash. Runs TFLite Micro. Prefer 38-pin for easier pin access.
MPU6050 / GY-521 IMU	2	199–290	400	6-axis accelerometer + gyroscope, I2C. Buy two: units arrive faulty often enough to matter, and a working pair gives the waist-vs-wrist placement ablation for free.
TP4056 charging module (with protection)	1	100	100	USB LiPo charging plus over-discharge protection. Buy the version <i>with</i> protection circuitry.
3.7 V LiPo cell, 1000 mAh	1	250	250	Portable power for the E4 battery-life measurement.
Active buzzer + LED + resistors	1 set	60	60	Alarm output. Active buzzer needs only a GPIO high, no PWM tone generation.
Perfboard, jumper wires, header pins	1 set	190	190	Breadboard prototype first, then solder to perfboard for the wearable build.
Subtotal			1,450	

Optional, from the remaining ~1,550 BDT

Component	BDT	Why you might want it
Elastic belt / velcro strap	150	Repeatable waist mounting. Inconsistent placement between trials shows up as noise in your results.
Small ABS project enclosure	200	Needed for the photograph in the paper. A breadboard photo undermines a deployment claim.
USB current meter	350	Direct power measurement for E4 rather than inferring from runtime.
Spare ESP32	450	Insurance. A dead board three days before submission is a project-ending event.
SD card module	250	Log raw sensor data on-device for your own small validation set.

8.1 Wiring

From	To	Note
MPU6050 VCC	ESP32 3V3	Do not put 5V on the ESP32's 3.3 V I2C lines
MPU6050 GND	ESP32 GND	
MPU6050 SDA	ESP32 GPIO 21	I2C data (default)
MPU6050 SCL	ESP32 GPIO 22	I2C clock (default)
MPU6050 AD0	GND or 3V3	Selects address 0x68 or 0x69. Required when running two IMUs on one bus.
Buzzer +	ESP32 GPIO 4	Through a transistor if the buzzer draws more than 12 mA
Buzzer –	ESP32 GND	
LiPo	TP4056 B+ / B–	
TP4056 OUT+	ESP32 VIN (or 5V pin)	Verify your board's regulator accepts 3.7 V input before relying on it

8.2 Sensor configuration

- **Accelerometer range:** ± 16 g. Fall impacts routinely exceed 8 g; a clipped peak is an unrecoverable loss of exactly the signal you care about.
- **Gyroscope range:** ± 2000 deg/s.
- **Sampling:** 50 Hz driven by a hardware timer interrupt. Do *not* use `delay(20)` in the main loop — the jitter accumulates and shifts window content relative to training.
- **I2C clock:** 400 kHz. The default 100 kHz leaves less headroom than you want at 50 Hz with two devices.

9. Deployment pipeline

Step 1 — Convert to TFLite with full integer quantisation

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_gen # ~200 training windows
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
open("model.tflite", "wb").write(converter.convert())
```

The representative dataset must come from the *training* split. Drawing it from test data leaks information and invalidates your results.

Step 2 — Verify before deploying

Run the quantised model in the Python TFLite interpreter on the full test set and compare against FP32. If accuracy dropped more than about two points, the problem is quantisation, not hardware. Fix it here — debugging this on the MCU is dramatically harder.

Step 3 — Convert to a C array

```
xxd -i model.tflite > model.h
```

Edit the generated file to mark the array `const` so it lands in flash rather than RAM.

Step 4 — Firmware structure

```
setup():
  init I2C at 400 kHz, configure MPU6050 (+/-16 g, +/-2000 dps)
  allocate tensor arena (start at 40 KB, tune down after measuring)
  load model from model.h, build interpreter
  start 50 Hz hardware timer

timer ISR (every 20 ms):
  read 6 raw values, push into ring buffer, set data-ready flag

loop():
  if 25 new samples since last inference:      # 0.5 s stride
    copy 100 x 6 window from ring buffer
    convert raw counts to g and deg/s
    apply the frozen normalisation constants
    quantise to int8 using the model's input scale and zero point
    invoke interpreter
    dequantise output
    if p(alert) or p(fall) > threshold:
      buzzer on, send WiFi alert, start cooldown
```

Step 5 — Instrument for measurement

Wrap the invoke call in `esp_timer_get_time()` and average over 1,000 inferences. Read the tensor arena high-water mark. These two numbers go directly into your results table and are what make C3 a real contribution rather than a claim.

Library options. `esp-tflite-micro` is the official Espressif port and gives the most control. `EloquentTinyML` is a thinner Arduino-friendly wrapper that is faster to get running. Start with whichever compiles first; switching later does not touch the model.

10. Results tables to fill

Table I — Within-dataset baseline, SisFall Enhanced

Baselines: 5-fold subject-grouped CV. Proposed model: full 38-fold LOSO. State the difference in the caption.

Model	Protocol	Params	Sensitivity	Specificity	Macro-F1	AUC
SMV threshold	5-fold grouped	—	—	—	—	—
SVM (hand-crafted features)	5-fold grouped	—	—	—	—	—
Random Forest	5-fold grouped	—	—	—	—	—
1D-CNN	5-fold grouped	—	—	—	—	—
CNN-LSTM	5-fold grouped	—	—	—	—	—
Proposed (separable CNN)	38-fold LOSO	—	—	—	—	—

Table II — Cross-dataset generalisation (the headline table)

Fold	Train	Test	F1 (none)	F1 (inst. norm)	F1 (CORAL)	Drop vs Table I
A	SisFall Enh. + KFall	FallAIID	—	—	—	—
B	SisFall Enh. + FallAIID	KFall	—	—	—	—
C	KFall + FallAIID	SisFall Enh.	—	—	—	—
D	All three	UMAFall	—	—	—	—

Table III — Pre-impact performance, both label sources

Label source	Threshold	Mean lead time (ms)	Std (ms)	Sensitivity	False alarms / hour ADL
KFall (primary)	0.5	—	—	—	—
	0.7	—	—	—	—
	0.9	—	—	—	—
SisFall Enhanced (corroborating)	0.5	—	—	—	—
	0.7	—	—	—	—
	0.9	—	—	—	—

Table IV — On-device deployment

Metric	FP32 (desktop)	INT8 (desktop)	INT8 (ESP32, measured)
Model size (KB)	—	—	—
Peak RAM (KB)	—	—	—
Inference latency (ms)	—	—	—
Macro-F1 on held-out set	—	—	—
Battery life (hours)	—	—	—

11. Ten-week timeline

Week	Objective	Deliverable that proves it is done
1	Acquire datasets and validate SisFall Enhanced	All four downloaded. Section 3.2 verification checklist passed and recorded. One notebook plotting a fall and an ADL from each. BibTeX file containing all dataset citations including both SisFall papers.
2	Freeze the preprocessing pipeline	Shared preprocessing module. All section 4.2 sanity checks passing, including visual inspection of the Enhanced alert intervals. Cached <code>.npz</code> uploaded to Kaggle. ESP32 + MPU6050 on a breadboard printing live values.
3	E1 baseline	Table I complete. Your model is competitive with published results, or you know precisely why not.
4-5	E2 cross-dataset	Table II complete including at least the instance-normalisation column. This is the core contribution — give it two full weeks.
6	E3 pre-impact, both label sources	Table III complete plus the lead-time-versus-false-alarm curve and a note on cross-source agreement.
7	Quantise and deploy	Model running on the ESP32. Buzzer fires on a deliberate drop onto a mattress.
8	E4 measurement and E5 ablations	Table IV complete with real measured numbers. Ablation figures generated.
9	Write	Full draft: abstract, introduction, related work, method, results, discussion, conclusion. All figures at publication resolution.
10	Revise and submit	Supervisor feedback incorporated. Formatting checked against the venue template. Code repository made public.

12. Risk register

Risk	Likelihood	Mitigation
SisFall Enhanced fails the verification checklist	Medium	Download the original SisFall in week 1 regardless. Fall back to original for training and rely on KFall alone for C2. Costs a contribution, not the project.
Enhanced pre-impact labels disagree with KFall	Medium	Report both separately and discuss. Annotation disagreement between two published sources is a legitimate finding, not a failure — but only if you frame it deliberately rather than discovering it in review.
A dataset host goes offline mid-project	Medium	Download all four in week 1, keep local plus cloud backups. The official SisFall link has been reported broken before; Kaggle mirrors exist for SisFall and KFall.
Cross-dataset drop is an axis-convention artifact	Medium	Run the section 4.2 orientation checks before drawing any conclusion. Document the rotation applied to each dataset.
Model will not fit the tensor arena	Medium	The window-length and sampling-rate ablations give three independent levers. Reducing to 1.0 s at 25 Hz cuts input size fourfold.
Kaggle quota exhausted	Low	Now unlikely at ~17 estimated GPU hours. The M4 provides overflow capacity if needed.
MPU6050 arrives faulty	Medium	Two units budgeted. Test both in week 2, not week 7.
Domain adaptation fails to recover the gap	Low-Medium	Still publishable as a rigorous negative result. "The gap is larger and harder to close than the literature implies" is a legitimate contribution — frame it that way rather than burying it.
Timeline slips	High	C1 and C3 alone constitute a complete paper. If time runs short, drop C2 and keep cross-dataset plus deployment.

13. Paper outline and venues

13.1 Section plan

- Introduction** — falls as a health burden; why wearable detection; the two gaps (untested generalisation, unverified deployment).
- Related work** — wearable fall datasets and their annotations; deep learning approaches; the saturation of within-dataset results; the small pre-impact literature; TinyML on MCUs.
- Datasets and preprocessing** — sections 2 to 4 condensed, including the UP-Fall exclusion rationale and the annotation-quality discussion.
- Proposed method** — architecture, quantisation strategy, domain adaptation.
- Experimental setup** — E1 to E5 protocols, the tiered cross-validation rationale, hardware description, metric definitions.
- Results** — Tables I to IV plus the lead-time curve and ablation figures.
- Discussion** — what the cross-dataset gap means for deployed systems; limitations (simulated falls, mostly young subjects, single placement, annotation subjectivity).
- Conclusion and future work.**

13.2 Candidate venues

Bangladeshi and regional IEEE conferences that regularly accept undergraduate work of this type: ICCIT, ICECE, ICAEEE, STI, and IEEE TENSYP. Verify current deadlines and templates on each conference site — do not rely on last year's dates. If results are strong, MDPI *Sensors* and IEEE *Access* are plausible journal targets, though both carry article processing charges worth confirming with your supervisor first.

13.3 Reproducibility checklist

- Public GitHub repository with preprocessing, training and firmware code.
- Frozen normalisation constants published as a JSON file.
- The `.tflite` model file released.
- Exact dataset versions and download dates recorded — especially important for the community-hosted Enhanced release.
- Random seeds fixed and stated.

14. Key references to obtain first

1. Sucerquia, A., López, J.D. and Vargas-Bonilla, J.F., "SisFall: A Fall and Movement Dataset," *Sensors*, 17(1):198, 2017. — the original recordings
 2. Musci, M., De Martini, D., Blago, N., Facchinetti, T. and Piastra, M., "Online Fall Detection using Recurrent Neural Networks," arXiv:1804.04976, 2018. Extended as "Online Fall Detection Using Recurrent Neural Networks on Smart Wearable Devices," *IEEE Transactions on Emerging Topics in Computing*, 9:1276–1289, 2020. — **the Enhanced annotations. Mandatory citation.**
 3. Yu, X., Jang, J. and Xiong, S., "A Large-Scale Open Motion Dataset (KFall) and Benchmark Algorithms for Detecting Pre-impact Fall of the Elderly Using Wearable Inertial Sensors," *Frontiers in Aging Neuroscience*, 13:692865, 2021.
 4. Saleh, M., Abbas, M. and Le Bouquin Jeannès, R., "FallAID: An Open Dataset of Human Falls and Activities of Daily Living for Classical and Deep Learning Applications," *IEEE Sensors Journal*, 2020.
 5. Casilari, E., Santoyo-Ramón, J.A. and Cano-García, J.M., "UMAFall: A Multisensor Dataset for the Research on Automatic Fall Detection," *Procedia Computer Science*, 110:32–39, 2017.
 6. Casilari, E. et al., "A Study on the Application of Convolutional Neural Networks to Fall Detection Evaluated with Multiple Public Datasets," *Sensors*, 2020. — the cross-dataset degradation evidence underpinning C1
 7. "TinyFallNet: A Lightweight Pre-Impact Fall Detection Model," *Sensors*, 23(20):8459, 2023. — closest prior work on C2 and C3; read before finalising your architecture
 8. Torti, E. et al., "Embedding Recurrent Neural Networks in Wearable Systems for Real-Time Fall Detection," *Microprocessors and Microsystems*, 71:102895, 2019. — the same group's MCU deployment using the Enhanced annotations; your closest deployment baseline
 9. "The Impact of the Accelerometer Sampling Rate on the Performance of Machine and Deep Learning Models in Wearable Fall-Detection Systems," *Sensors*, 2025. — supports your sampling-rate ablation
-

Prices, dataset URLs and access conditions verified August 2026. Confirm dataset availability and conference deadlines before committing to the schedule. Reference 8 is worth reading early — it is the same research group that produced the Enhanced annotations and it deploys to a microcontroller, making it simultaneously your closest related work and your most useful implementation guide.